PYTHON WINTER SPRINT 2026

CODING COUNCIL
a student led tech council

W3B JMI

Core Problem of 8th Jan 2026 : Vault OS – Your Personal Password Manger

Each Day have one constant theme and three different tracks : Foundation Track especially due to W3B Collaboration, Core Track for those following since DAY-1 and the Project Track for those who have their basics of Python Cleared, they will make these end-to-end projects

FOUNDATION TRACK (FOR ABSOLUTE BEGINNERS AND NEW JOINERS)

[Expected Time to Finish – 2hrs 20 minutes]

Resource: https://www.youtube.com/watch?v=UrsmFxEIp5k from 1hr 40 minutes till 2hr 53 minutes

Core Topic: Lists, Tuples, Dictionary & Sets

🧠 Problem Statement

Write a Python program that collects and organizes basic user account data using lists, tuples, dictionaries, and sets.

The program should do the following:

Take input from the user:

- Username
- Platform name (e.g. GitHub, Instagram, Gmail)
- Account type (Personal / Work)

Store the data as follows:

- Store all usernames in a list
- Store (platform, account type) together as a tuple
- Store username → platform mapping in a dictionary
- Store unique account types in a set

Sample Input:

```
Enter username: ali_khan
Enter platform: github
Enter account type: work
```

Sample Output:

```
--- ACCOUNT DATA SUMMARY ---
Usernames        : ['ali_khan']
Platform & Type  : ('github', 'work')
User Mapping     : {'ali_khan': 'github'}
Account Types    : {'work'}
```

Beginners Please ask in Group if you get to encounter any problem, any kind of problem in this, or just DM me
Every One was once a beginner !!

CODING COUNCIL
a student led tech council

PYTHON WINTER SPRINT 2026

W3B JMI

CORE TRACK (FOR STUDENTS WHO HAVE BEEN LEARNING FROM DAY 1)

Core Topic: File I/O

Resource: https://www.youtube.com/watch?v=UrsmFxEIp5k from 5hr 54 minutes till 6hr 42 minutes

[Expected Time to Finish - 2hrs 10 minutes]

📘 Core Track Project (8th Jan)

Vault Lite – File-Based Credential Manager

This is a learning-focused, simplified version of Vault OS to introduce File I/O + structured logic.

🎯 Objective

Build a basic CLI credential manager that uses a text/JSON file to:

- Save data
- Read data
- Update data

This project teaches how programs store and retrieve data from files, which is a core backend skill.

🧩 Features (Simplified on Purpose)

✅ What Core Track MUST Implement

1. Main Menu (Loop-based)
   a. View saved credentials
   b. Add new credential
   c. Update a credential
   d. Exit
2. File Storage
   - Use a file named vault_data.json
   - Data must persist after program restart
3. Credential Format
   - Website
   - Username
   - Password
4. Add Credential
   - Take inputs
   - Save them to file
5. View Credentials
   - Read file
   - Display all stored entries
6. Update Credential
   - Show numbered list
   - Allow updating password only

❌ What Core Track Does NOT Need

- No animations
- No screen clearing
- No delete option
- No complex error handling
- No encryption

Sample Run:

Program Start

```
=== Vault Lite ===
1. View credentials
2. Add credential
3. Update credential
4. Exit

Enter choice:
```

Add Credential

```
Enter choice: 2

Website: github
Username: ali_khan
Password: pass123

Credential saved successfully.
```

View Credentials

```
Enter choice: 1

Saved Credentials:
1. ali_khan @ github
```

Update Credential

```
Enter choice: 3

1. ali_khan @ github
Enter number to update: 1
Enter new password: newpass456

Password updated successfully.
```

Exit

```
Enter choice: 4
Exiting Vault Lite.
```

vault_data.json (Example)

```
{
    "github_ali_khan": "newpass456"
}
```

PYTHON WINTER SPRINT 2026

PROJECT TRACK (FOR ADVANCED PARTICIPANTS, MINI PROJECT)

[Expected Time to Finish – 3-4 hours]

🎯 Objective

Build a command-line password manager that allows users to store, view, update, and delete credentials securely using a JSON-based local database.

This project simulates how real-world CLI tools manage persistent data and user workflows.

🧩 Core Features to Implement

Interactive Main Menu

Create a looping CLI menu with the following options:

- Access stored credentials
- Store a new credential
- Update an existing credential
- Remove a saved credential
- Exit the application

The program should continue running until the user chooses to exit.

Persistent Storage (JSON)

- Use a database.json file to store credentials.
- Data must persist even after restarting the program.
- Handle cases where the file is missing or empty.

Credential Structure

Each credential should store:

- Website name
- Username
- Password

Use a unique key format combining website and username.

Add New Credential

- Ask user for website, username, and password.
- Save the data into database.json.
- Confirm successful storage.

View Stored Credentials

- Display all saved credentials in a readable format.
- Handle the case where no data exists.

Update Existing Credential

- Show numbered list of stored credentials.
- Let user choose which entry to update.
- Replace the old password with a new one.

Delete Credential

- Show numbered list of credentials.
- Allow user to remove a selected entry.
- Update the JSON file accordingly.

PYTHON WINTER SPRINT 2026

PROJECT TRACK (FOR ADVANCED, SYSTEM-LEVEL PROBLEM SOLVING)

User Experience Enhancements

- Clear the screen between operations.
- Display meaningful messages for:
  - Invalid input
  - Empty database
  - Successful actions
- Add slight delays or styled output for better CLI feel (optional).

🛠️ Technical Constraints

- Language: Python
- Interface: Command Line (CLI)
- Storage: JSON file
- Use functions for:
  - Menu handling
  - Data operations
- Use loops to keep the program running.

📂 Expected File Structure

```
vault-os/
│
├── main.py
└── database.json
```

Sample Run:

Program Start

```
Welcome to Vault OS 1.0 — Your Personal Password Commander
---------------------------------------------------------
Choose an operation:
1. Access stored credentials
2. Store a new credential
3. Update existing credential
4. Remove a saved credential
5. Shut down Vault OS

--> Vault OS command:
```

Option 2: Store a New Credential

```
--> Vault OS command: 2

Website: github
Username: arshad_ali
Password: mySecurePass123

Saving to Vault...
Operation completed. Returning to menu...
```

Option 1: Access Stored Credentials

```
--> Vault OS command: 1

Decrypted credentials:
arshad_ali's github -> mySecurePass123

Returning to Vault OS menu...
```

Option 3: Update Existing Credential

```
--> Vault OS command: 3

1. arshad_ali's github account -> mySecurePass123
Enter number to update: 1
New password for arshad_ali's github account: newStrongPass456

Credential updated successfully.
```

Option 4: Delete Credential

```
--> Vault OS command: 4

1. arshad_ali's github account -> newStrongPass456
Enter number to delete: 1

Entry deleted. Returning to menu...
```

Empty Database Case

```
--> Vault OS command: 1

Vault OS database is empty, Commander.
```

Option 5: Exit Vault OS

```
--> Vault OS command: 5

Shutting down Vault OS... Stay secure, Commander!
```

database.json (After Adding Credentials)

```
{
    "github_arshad_ali": "newStrongPass456"
}
```